

Dual-Track TDD

Cursor AI 활용 소프트웨어 개발

하루 8시간 집중 과정 | PART 3-6 완전 실습

하루 8시간 — 수업 시간표

09:00 – 10:30

SECTION 1

TDD 개념 & Dual-Track 구조

12:00 – 13:00

점심 Break

휴식

14:30 – 16:00

SECTION 4

Golden Master & 회귀 안전망

17:30 – 18:30

SECTION 6

QA / 커버리지 / 최종 보고서

10:30 – 12:00

SECTION 2

RED 단계 — 실패 테스트 설계

13:00 – 14:30

SECTION 3

GREEN 단계 — 최소 구현

16:00 – 17:30

SECTION 5

Refactoring — 구조 개선

18:30 – 19:00

WRAP-UP

회고(KPT) & 현업 적용 계획

SECTION 1

TDD 개념 & Dual-Track 구조

왜 Dual-Track TDD인가?

Track A — UI / Boundary

- UI/Boundary 계층을 독립적으로 검증
- 입력 계약 위반을 Boundary에서 차단
- 모킹(Mock)으로 Domain 격리 가능
- E001~E007 Error Contract 보호
- 실제 사용자 시나리오와 직결

Track B — Domain / Logic

- Domain 알고리즘을 순수 함수로 검증
- Mock 없이 실제 Entity 호출
- Invariant(I1~I11) 단위 보장
- Control 오케스트레이션 검증
- 비즈니스 로직 독립 보장

ECB 아키텍처 — 책임 분리

Boundary (E)

- 입력 검증 (E001~E005)
- FailureResponse 반환
- 출력 직렬화 int[6]
- Screen UI 포함
- Domain 알고리즘 금지

Control (C)

- 유스케이스 오케스트레이션
- locate → find → solve
- E006/E007 예외 매핑
- I/O 검증 금지
- UI 문자열 포함 금지

Entity (E)

- Value Object / Domain Service
- 불변식(Invariant) 보호
- 순수 함수 패턴
- Boundary/Control import 금지
- 4×4=34 Magic Constant

스켈레톤 코드(Skeleton Code) — 뼈대 코드

```
# MagicSquareSolver — 스켈레톤 예시
from typing import List, Tuple

class MagicSquareSolver:
    def find_blank_positions(self, grid: List[List[int]]) -> List[Tuple[int,int]]:
        """0의 위치를 row-major 순으로 반환한다."""
        pass # 🟡 아직 구현 없음

    def find_missing_numbers(self, grid: List[List[int]]) -> List[int]:
        """1~16 중 빠진 숫자를 오름차순으로 반환한다."""
        pass # 🟡 아직 구현 없음

    def solve(self, grid: List[List[int]]) -> List[int]:
        """마방진을 완성한다. int[6] 형식으로 반환."""
        pass # 🟡 아직 구현 없음
```

💡 구조만 정의 → 테스트가 먼저 실패해야 RED가 성립된다!

Git 브랜치 전략 — Dual-Track TDD

브랜치	단계	핵심 규칙
main	릴리스	GREEN 확인 후 머지 / 태그 부여
develop	통합	feature → develop PR 머지 (리뷰 필수)
feature/dual-track-tdd	RED 단계	실패 테스트 작성만 — 구현 코드 금지
stabilize/green	GREEN 단계	최소 구현 / 1커밋 = RED 1묶음
refactor/refactor	Refactoring	구조 개선 / 외부 계약 불변

Magic Square 4x4 — 문제 정의

Magic Square 4x4 완전 정의 (PRD §8.1)

- 입력: 4x4 int[][] — 0=빈칸 (정확히 2개), 값은 0 또는 1~16, 중복 금지
- 출력: int[6] = [r1, c1, n1, r2, c2, n2] — 좌표는 1-index
- 마방진 상수: 모든 행·열·대각선 합 = 34 (Magic Constant)
- 검증 순서(short-circuit): null → size → empty count → value range → duplicate
- 조합 전략: 작은 수 → 첫 빈칸 (Step A) → 실패 시 역순 (Step B)
- 예시 입력: [[16,2,3,13],[5,11,10,8],[9,7,0,12],[4,14,15,0]]
- 예시 출력: [3,3,1,4,4,6] 또는 [3,3,6,4,4,1]

SECTION 2

RED 단계 — 실패 테스트 설계

RED 단계 핵심 원칙

Track A — UI/Boundary RED 테스트

- 입력 계약 검증 테스트
- 4x4 아닌 입력 → 예외 (E001)
- 빈칸 ≠ 2개 → 예외 (E002)
- 값 범위 위반 → 예외 (E004)
- 중복 숫자 → 예외 (E005)
- 반환 배열 길이 = 6 (U-OUT-01)
- 반환 좌표 = 1-index (U-OUT-02)

Track B — Domain/Logic RED 테스트

- 도메인 로직 단위 테스트
- find_blank_coords() — row-major 2개
- find_not_exist_nums() — 오름차순 2개
- is_magic_square() — 행·열·대각선 = 34
- solution() — 조합 시도 + 1-index 반환
- Domain Mock 절대 금지
- 실제 Entity 호출로 검증

RED 단계 절대 제약 조건

RED Phase — 엄격한 금지 사항

- ⚠️ 구현 코드(production code) 작성 절대 금지
- ⚠️ GREEN-REFACTOR 단계 진입 금지
- ⚠️ 모든 테스트는 현재 실패(FAIL/ERROR) 상태여야 함
- ⚠️ Domain은 Mock으로만 가정 (Track A에서만 허용)
- ⚠️ `pytest.fail()` 한 줄로만 의도적 실패 표현
- ⚠️ `skip`, `xfail`, `assert` 완화 금지
- ⚠️ Report/08 기존 테스트 수정·삭제 금지

Track A — UI/Boundary RED 설계표

Test ID	Given	Then (기대값)	Expected RED Failure
U-IN-01	grid=None	E003 INVALID_NULL	ModuleNotFoundError
U-IN-02	grid=3×4	E001 INVALID_SIZE	AssertionError
U-IN-03	빈칸 0개	E002 INVALID_BLANKS	AssertionError
U-IN-04	값=-1 또는 17	E004 INVALID_RANGE	AssertionError
U-IN-05	중복 숫자	E005 INVALID_DUPLICATE	AssertionError
U-OUT-01	유효 입력 G1	len(result)==6	pytest.fail() RED
U-OUT-02	유효 입력 G1	좌표 1~4 범위	pytest.fail() RED
U-FLOW-02	grid=None	execute() 0회 호출	pytest.fail() RED

Track B — Domain/Logic RED 설계표

Test ID	대상 함수	Given/Then	Invariant
D-LOC-01	find_blank_coords()	G1 → [(2,2),(3,3)]	I6 row-major
D-MIS-01	find_not_exist_nums()	G1 → [7,10] 오름차순	I7,I11
D-VAL-01	is_magic_square()	G0 완전 → True	I1~I5
D-VAL-02	is_magic_square()	행 합 불일치 → False	I1
D-VAL-03	is_magic_square()	열 합 불일치 → False	I2
D-VAL-04	is_magic_square()	대각 불일치 → False	I3
D-SOL-01	solution()	G1 Step A 성공	I8
D-SOL-02	solution()	G2 Step B 성공	I9

RED 스켈레톤 — pytest.fail() 패턴

```
# tests/entity/test_d_loc_01_empty_cell_locator.py
import pytest

class TestEmptyCellLocator:
    """D-LOC-01: G1 → row-major 기준 (2,2), (3,3) 반환"""

    @pytest.fixture
    def grid_g1(self):
        return [[16,3,2,13],[5,0,11,8],[9,6,0,12],[4,15,14,1]]

    def test_d_loc_01_blank_coords_row_major(self, grid_g1):
        # Given: G1 격자 (0이 2개: [1][1], [2][2])
        # When: find_blank_coords(grid_g1) 호출
        # Then: [(2,2),(3,3)] 반환 (1-index, row-major)
        pytest.fail("RED: D-LOC-01 - 구현 없음, 의도적 실패")
```

💡 `pytest.fail()` 한 줄 → 구현 없이 RED 상태 유지

테스트 환경 설정 — 가상환경 & 실행

1. 프로젝트 루트에서 가상환경 생성

```
python -m venv .venv
```

2. 가상환경 활성화 (Git Bash / WSL)

```
source .venv/Scripts/activate
```

3. 의존성 설치

```
pip install pytest pydantic pytest-cov
```

4. Dual-Track 실행

```
python -m pytest tests/boundary/ -v # Track A
```

```
python -m pytest tests/entity/ -v # Track B
```

5. 기대 결과 (RED 단계)

```
# ModuleNotFoundError: No module named 'boundary' ← 의도한 실패 ✓
```



RED 단계: collection ERROR도 정상! 구현 없음을 확인한 것

SECTION 3

GREEN 단계 — 최소 구현

GREEN 단계 핵심 원칙 — 1커밋 = RED 1묶음

GREEN Phase — 최소 구현 원칙

- 1단계: RED 테스트 1개 선택 → 의도한 실패 확인
 - 2단계: 선택한 테스트를 통과할 최소 코드만 작성
 - 3단계: 동일 테스트 재실행 → PASS 확인
 - 4단계: git commit (Conventional Commit 형식)
 - 5단계: 다음 RED 테스트로 이동 (1번으로 반복)
-
- ⚠ 테스트 약화·삭제·skip/xfail 금지
 - ⚠ 하드코딩·매직 넘버 추가 금지
 - ⚠ REFACTOR 이번 커밋에서 금지
 - ⚠ 한 커밋에 모든 RED 해결 금지

GREEN 1단계 — AC-FR-01-01 최소 구현

```
# src/boundary/schemas.py
from pydantic import BaseModel
class ErrorDetail(BaseModel):
    code: str
    message: str
class FailureResponse(BaseModel):
    type: str = 'ERROR'
    error: ErrorDetail

# src/boundary/input_validator.py
from boundary.schemas import FailureResponse, ErrorDetail
class InputValidator:
    def validate(self, grid):
        if grid is None:
            return FailureResponse(error=ErrorDetail(code="INVALID_SIZE", message="Grid must
be 4x4.))
        raise NotImplementedError # 다른 케이스는 다음 커밋
```

💡 grid=None 분기만 추가 — 다른 입력은 다음 GREEN 커밋으로!

GREEN TDD Cycle — Turn별 로드맵

Turn	대상 테스트	Track	최소 구현 범위
Turn 1	AC-FR-01-01 grid=None	A	InputValidator.validate(None) → FailureResponse
Turn 2	size 3건 parametrize	A	4x4 크기 검증 분기 추가
Turn 3	UIBoundary grid=None + resolve 0회	A	UIBoundary.solve() + Control stub
Turn 4	G1 입력 통과 FR-01 완성	A	빈칸·범위·중복 검증 완성
Turn 5	D-LOC-01 (find_blank_coords)	B	EmptyCellLocator 서비스 구현
Turn 6	D-MIS-01 (find_not_exist_nums)	B	MissingNumberFinder 구현
Turn 7	D-VAL-01~06 (is_magic_square)	B	MagicSquareValidator 구현
Turn 8	D-SOL-01~04 (solution)	B	TwoCellSolver G1/G2/G3

GREEN 후 커버리지 확인

1. Boundary 커버리지 (목표: $\geq 85\%$)

```
python -m pytest tests/boundary/ \
    --cov=src/boundary --cov-report=term-missing
```

2. Domain(Entity) 커버리지 (목표: $\geq 95\%$)

```
python -m pytest tests/entity/ \
    --cov=src/entity --cov=src/control --cov-report=term-missing
```

3. 전역 커버리지 (목표: $\geq 80\%$)

```
python -m pytest --cov=src --cov-report=html
```

4. HTML 리포트 열기

```
start htmlcov/index.html
```



RED 단계: Entity만 GREEN → htmlcov 생성 가능 | Boundary RED → htmlcov 미생성

SECTION 4

Golden Master — 회귀 안전망 구축

Golden Master(GM) — 회귀 테스트 기법

Golden Master — Refactoring 안전장치

- 정의: 현재 출력을 기준 파일에 저장 → Refactoring 후에도 동일 출력 유지 자동 검증
- 구축 시점: GREEN 단계 완료 후 / Refactoring 시작 직전
- 비교 방식: expected vs actual → unified diff 출력 후 FAIL
- approve 패턴: 기준 파일 없으면 자동 생성 / 있으면 비교
- GM-TC-01: 정상 조합 성공 → [3,3,1,4,4,6]
- GM-TC-02: reverse 조합 성공 → [3,3,6,4,4,1]
- GM-TC-03: INVALID_BLANK_COUNT (E002)
- GM-TC-04: DUPLICATE_NUMBER (E005)
- GM-TC-05: NO_VALID_MAGIC_SQUARE (E006)

Golden Master — 핵심 명령어

1. 기준 파일 생성 (최초 1회)

```
python scripts/generate_golden_master.py
```

→ tests/golden_master_expected.txt 생성

2. 회귀 검증 (Refactoring 매 커밋 후)

```
python -m pytest tests/test_gm_01_magic_square_golden_master.py -v
```

→ matched: PASS | diff 발생: FAIL

3. CI용 비교만 (파일 수정 없음)

```
python scripts/generate_golden_master.py --check
```

→ 불일치 시 exit code 1

4. 의도적 변경 후 approve (ISS-012-01 문서화 필수)

```
pytest tests/test_gm_01_magic_square_golden_master.py --approve-golden -v
```



GM diff = 회귀 or 의도 변경 — 반드시 근거 기록 후 approve!

Golden Master — 보호하는 불변 계약

GM 규칙	보호 내용	위반 시
GM-01	int[6] 출력 형식 보존	FAIL — 포맷 회귀
GM-02	row-major 첫 빈칸 규칙	FAIL — 탐색 순서 회귀
GM-03	1-index 좌표 규칙	FAIL — 인덱스 회귀
GM-04	작은 수→첫 빈칸 (Step A 우선)	FAIL — 조합 순서 회귀
GM-05	reverse fallback (Step B)	FAIL — fallback 회귀
GM-06	E001~E007 Error Contract	FAIL — 오류 계약 회귀
GM-07	Magic Constant = 34	FAIL — 마방진 상수 회귀

SECTION 5

Refactoring — 구조 개선 (계약 불변)

Refactoring 절대 원칙

REFACTOR Phase — Safe Refactoring 원칙

- 외부 계약(입력/출력/예외 타입·메시지) 절대 변경 금지
- 매 커밋: pytest 전체 PASS + GM-1 matched 유지
- Change Budget: 수정 파일 ≤ 3 개 / 신규 클래스 ≤ 1 개 / 신규 메서드 ≤ 3 개
- 기능 추가·설계 확장·speculative abstraction 금지
- 하드코딩·매직 넘버 추가 금지 (MagicConstant SSOT 사용)
- ECB 역방향 import 금지 (entity $\rightarrow$ boundary/control)
- Phase 0 Gate: RED 0건 + GM matched + 전체 GREEN 필수

Refactoring 후보 — 코드 스멜 분류

ID	Track	대상	기법	우선순위
RF-01	UI	ValidationResult 도입	NotImplementedError 제거	P0
RF-02	UI	E006 ErrorMapper	예외→Envelope 매핑	P0
RF-03	Control	SolutionResult.values SSOT	중복 제거	P0
RF-04	Control+Entity	locate/find 중복 정리	Extract Method	P0
RF-05	UI	_failure() 추출	DRY — FailureResponse 반복	P1
RF-06	Screen	ResultPresenter 분리	SRP 적용	P1
RF-07	Entity	sumRow/sumCol/sumDiag	Extract Method	P2
RF-08	Entity	Coordinate VO 통일	Value Object 패턴	P2

Refactoring — 1커밋 Safe Refactor 절차

```
# Step 0: 기준선 확인 (필수)
python -m pytest tests/ -v
python -m pytest tests/test_gm_01_magic_square_golden_master.py -v
# → 전체 GREEN + GM matched 아니면 중단

# Step 1: 목표 1개 선언 (예: RF-01 - ValidationResult 도입)
# Step 2: 보호 테스트 점검 (연결된 test ID 확인)
# Step 3: Safe Refactor 수행 (Change Budget 내)

# Step 4: 회귀 검증
python -m pytest tests/ -v
python -m pytest tests/test_gm_01_magic_square_golden_master.py -v
# → 전체 PASS + matched 확인

# Step 5: Conventional Commit
# refactor(boundary): replace NotImplementedError with ValidationResult
```

💡 GM diff 발생 = 실패! 의도적 변경 시 ISS-012-01 문서화 후 approve

Cursor AI — Refactoring 전 Ask 모드 분석

Cursor AI Ask 모드 활용 — 분석 선행 후 실행

- Step 1: 기준선 확인 → 테스트 존재 여부 & GREEN 상태 점검
 - Step 2: 코드 스멜 탐지 → 중복·긴 함수·매직 넘버·책임 혼재
 - Step 3: ECB 책임 분리 분석 → 레이어 위반 파악
 - Step 4: SRP/SOLID 위반 점검 → 함수 다중 역할 탐지
 - Step 5: 종합 리팩토링 계획서 → 우선순위 로드맵 생성
-
- 핵심: Ask 모드는 코드 수정 없이 분석만!
 - Agent 모드 전환 후 1커밋 1목표 실행

SECTION 6

QA · 커버리지 · 최종 보고서

테스트 커버리지 목표 (P-09)

레이어	게이트 기준	측정 명령어	판정
Domain Logic (entity+control)	$\geq 95\%$	<code>pytest --cov=src/entity --cov=src/control</code>	NFR-01
Boundary (전체)	$\geq 85\%$	<code>pytest --cov=src/boundary</code>	NFR-02
전역 (src/)	$\geq 80\%$	<code>pytest --cov=src</code>	NFR-03
REFACTOR Gate	RED 0건 + 전역 $\geq 80\%$ + GM matched	—	진입 조건

최종 작업 보고서 구조 (P-10)

Report/NN.MagicSquare_[단계]_Report.md | Prompting/NN_Transcript.md

- 섹션 1: 작업 개요 — TDD Phase, Turn별 요약, 세션 워크플로
- 섹션 2: 완료 To-Do — TASK-ID × RED Test ID × ECB Layer × 상태
- 섹션 3: RED 단계 결과 — 테스트 목록 + pytest 실패 증거
- 섹션 4: GREEN 단계 결과 — PASS 목록 + 커밋 메시지 이력
- 섹션 5: REFACTOR 결과 — Wave/커밋 ID + 회귀 확인
- 섹션 6: 커버리지 현황 — 레이어별 실측값 + 판정
- 섹션 7: 미완료 항목 & 다음 단계 구체 액션
- 섹션 8: 이슈 & 결함 목록 — ISS/DEF ID + 근본 원인

SECTION WRAP

회고(KPT) & 현업 적용 계획

18:30 – 19:00 (30분)

KPT 회고 — Keep / Problem / Try

Keep

- Dual-Track 병렬 설계
- 1커밋 = 1 RED 원칙
- ECB 책임 분리 유지
- Golden Master 구축
- Cursor AI Ask 모드 활용

Problem

- RED → GREEN 경계 혼용
- 프롬프트 엔지니어링 부족
- AI 제안 검증 능력 미흡
- 커버리지 목표 미달
- ECB 레이어 책임 혼재

Try

- AC ID 명시 관행 강화
- GM approve 절차 문서화
- 도메인 Mock 금지 철저
- SSOT 상수 선행 정의
- Phase 0 Gate 자동화

현업 적용 계획 — 팀 차원 액션 아이템

항목	내용	책임자	일정	성공 기준
브랜치 전략	Dual-Track 브랜치 규칙 팀 공유	Tech Lead	1주 내	PR 규칙 문서화
커버리지 Gate	CI/CD에 80% 하한선 자동 적용	DevOps	2주 내	파이프라인 통과
.cursorrules	프로젝트별 TDD 규칙 파일 생성	팀 전체	즉시	리뷰에서 준수 확인
Golden Master	핵심 API 출력 기준 파일 관리	QA 담당자	3주 내	GM 회귀 0건
Cursor AI 활용	Ask 모드 → Agent 모드 워크플로	개발자 전원	4주 내	커밋당 RED 확인

Dual-Track TDD 전체 흐름 — 한눈에 보기

단계	브랜치	핵심 작업	금지 사항
① 브랜치 전략	main	개발 방법론 결정	코드 작성
② RED	feature/dual-track-tdd	UI+Domain 실패 테스트 설계	구현/GREEN/REFACTOR
③ GREEN	stabilize/green	RED 1묶음씩 최소 구현 → PASS	REFACTOR / 하드코딩
④ REFACTOR	refactor/refactor	외부 계약 불변 / 구조 개선	계약 변경 / 기능 추가
⑤ QA/커버리지	refactor/refactor	Domain 95%+ / Boundary 85%+	커버리지 기준 미달
⑥ PR → main	develop / main	리뷰 후 머지 / 릴리스 태그	리뷰 없는 머지

감사합니다

오늘 배운 것: RED → GREEN → REFACTOR → QA
계약을 지키며, 테스트가 보호하는 코드를 작성하세요.